

DPA-DP: layer-freezing sweep and privacy budget allocation

This work investigates how to apply differential privacy to federated learning for disaster prediction, using a layer-freezing sweep to determine which pretrained layers should remain trainable during fine-tuning, and a privacy budget allocation strategy (DPA-DP) that adapts noise levels between normal and disaster operating conditions.

Extended layer-freezing sweep

With the previously identified GroupNorm fix in place, the MobileNetV3 sweep on CIFAR-10 was extended to 40 rounds to observe longer-run convergence behavior across the layer-freezing configurations.

Disaster dataset baseline

MobileNetV3 was evaluated on the AIDER disaster-imagery dataset (federated, 10 clients, privacy disabled) to establish the best achievable accuracy on this more realistic but smaller and noisier dataset, ahead of introducing DP-SGD.

DPA-DP privacy allocation

An initial experiment simulated a realistic operational scenario - a long period of normal conditions followed by a short disaster period - comparing a standard fixed privacy budget against DPA-DP's adaptive allocation. This is an early test of the core DPA-DP idea and establishes a working pipeline for measuring the privacy/accuracy tradeoff between the two approaches.

Next steps

Extend the disaster-scenario simulation to a larger dataset. Refine the privacy budget allocation. Complete the text-modality (DistilBERT, MobileBERT) analysis in parallel.

```
In [1]: import pandas
import matplotlib.pyplot as plt

def flwr_accuracy(file, start=0, key="accuracy"):
    df = pandas.read_csv(file + ".csv")
    plt.figure(figsize=(8,4))
    for id_val, g in df.groupby("id", sort=False):
        g = g.sort_values("round").tail(len(g) - start)
        plt.plot(g["round"], g[key], label=str(id_val))

    plt.xlabel("round")
    plt.ylabel(key)
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.tight_layout()
    plt.show()
```

```
In [2]: import opacus.accountants
import matplotlib.pyplot as plt

noise_multiplier_1 = 1.1
noise_multiplier_2 = 1.1
noise_multiplier_switch = 90
rounds = 100
delta = 1e-5
n_samples = 50000
batch_size = 32

def epsilon_curve(accountant):
    n_batches = (n_samples + batch_size - 1) // batch_size
    sample_rate = 1 / n_batches
    epsilons = [0]
    for round_idx in range(rounds):
        noise_multiplier = noise_multiplier_1 if round_idx < noise_multipl
        for _ in range(n_batches):
            accountant.step(noise_multiplier=noise_multiplier, sample_rat
            epsilons.append(float(accountant.get_epsilon(delta=delta)))
    return epsilons

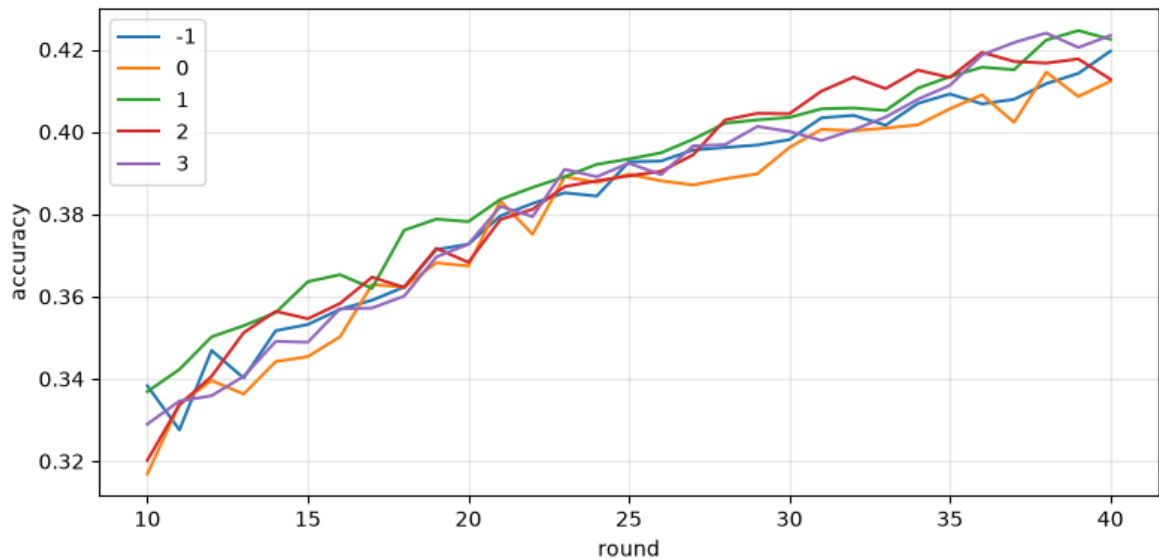
def plot_epsilon(data):
    plt.figure(figsize=(12,4))
    for name, arr in data.items():
        plt.plot(arr, label=name)
    plt.xlabel("round")
    plt.ylabel("epsilon")
    plt.grid(True, alpha=0.3)
    plt.legend()
    plt.show()
```

MobileNetV3, accuracy by trainable layers

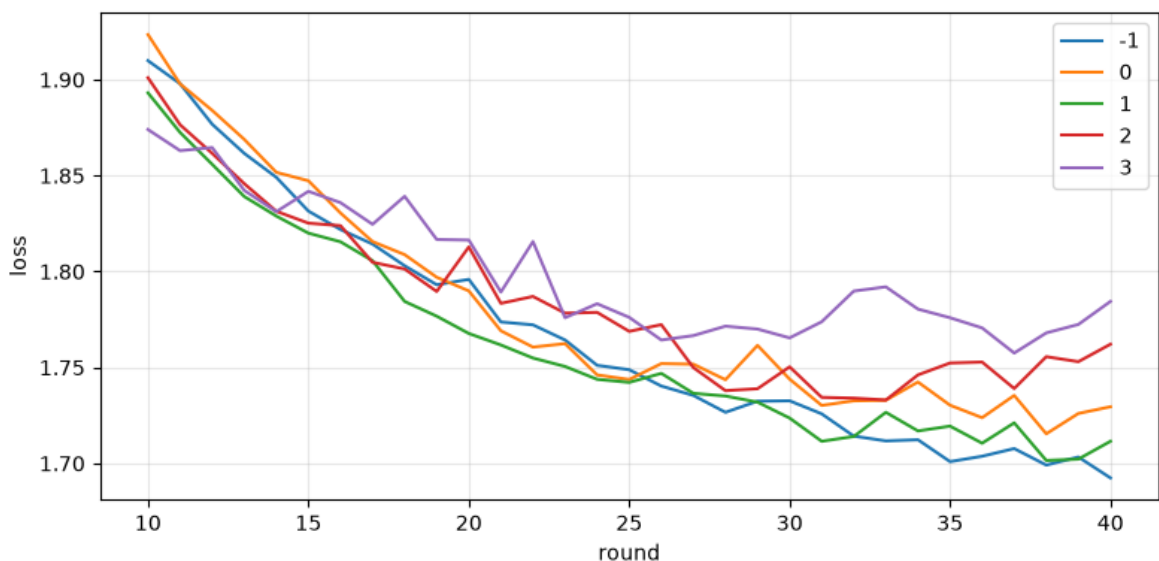
noise-multiplier: 1.1, max-grad-norm: 1.0, learning-rate: 0.01, batch-size: 32
The number of trainable weights is 22,362 (-1), 613,210 (0), 668,506 (1), 960,106 (2), and 1,251,706 (3) out of a total of 1,528,106.

In this experiment, we tested the accuracy of MobileNetV3 depending on the ratio of frozen and unfrozen layers. As it seems, on the CIFAR-10 dataset, adding more layers to the trainable pool does not bring significantly better accuracy, because the pretrained weights of the backbone perform well on the standard dataset. However, the growing loss of the higher layers shows a slight deterioration of the model, as the pretrained backbone is being exposed to the Opacus noise.

```
In [3]: flwr_accuracy("random_seed_layers", 10)
```



```
In [4]: flwr_accuracy("random_seed_layers", 10, "loss")
```



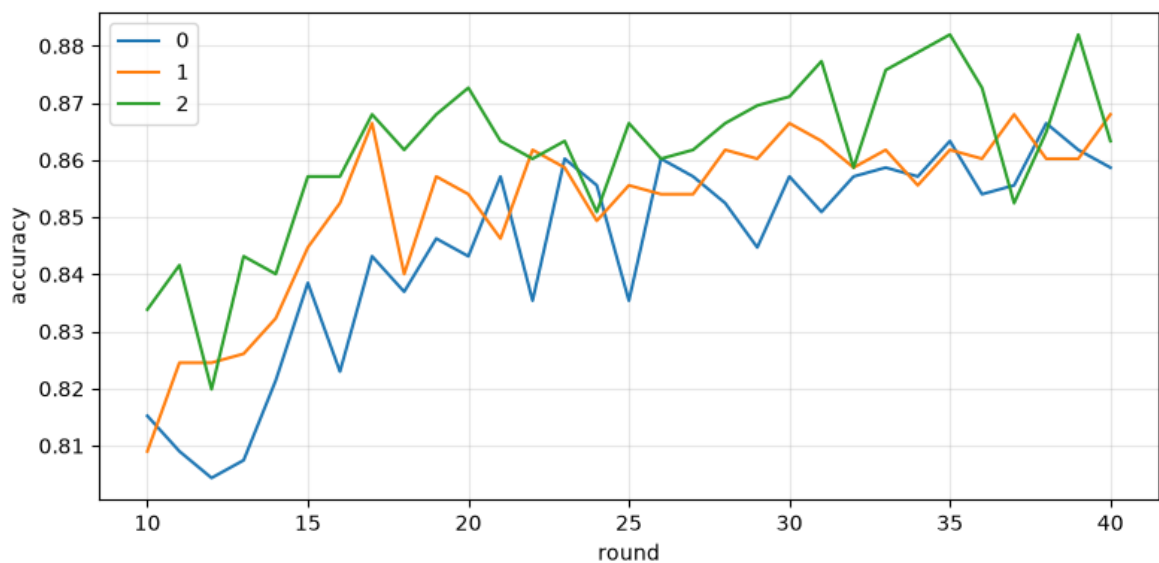
MobileNetV3, AIDER dataset, privacy disabled

noise-multiplier: 0, learning-rate: 0.1, batch-size: 32

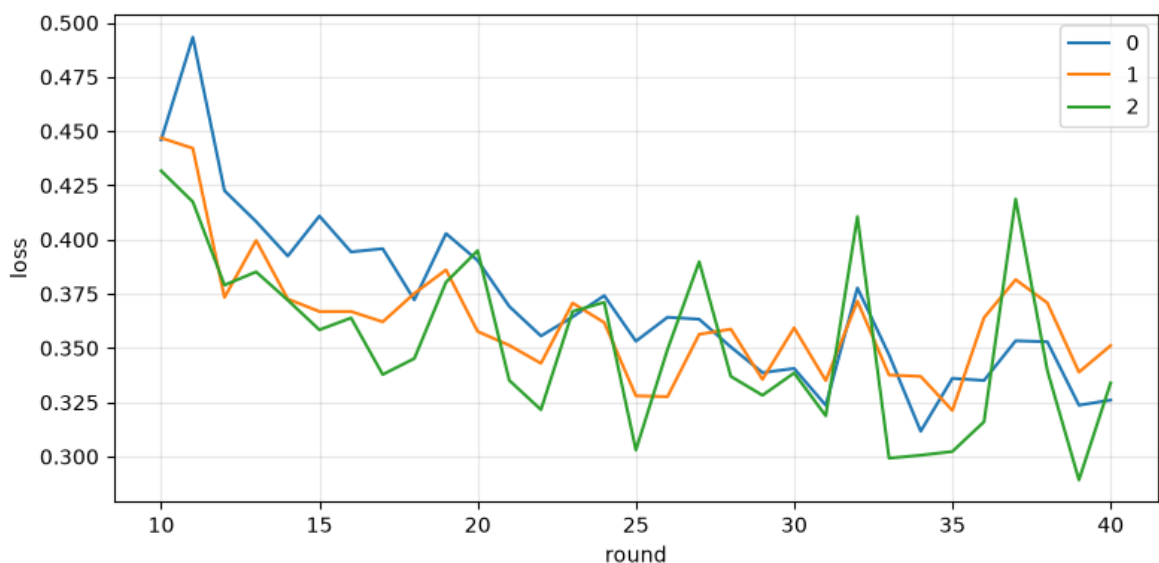
AIDER is less curated compared to CIFAR, it contains only 6,433 images, and some disaster images are indistinguishable from normal ones. However, it is the best available state-of-the-art dataset for disaster scenarios. This test was performed to estimate the best possible accuracy that can be achieved on the 10-client Flower federated setup with privacy noise switched off.

Frozen and unfrozen backbones show accuracy around 0.86, with more trainable weights showing a slightly better result.

```
In [5]: flwr_accuracy("aider_clear", 10)
```



```
In [6]: flwr_accuracy("aider_clear", 10, "loss")
```



DPA-DP privacy budget allocation

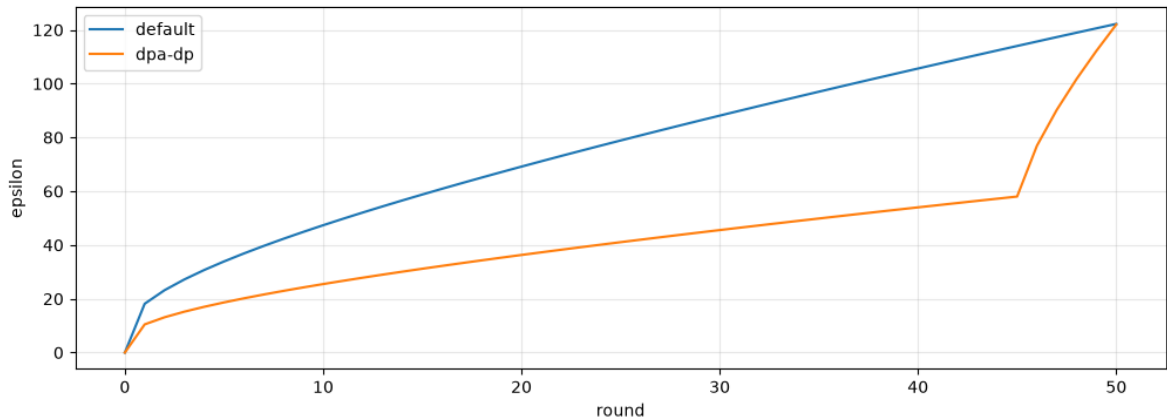
tune_layers: 0 (frozen backbone, trainable classifier), max-grad-norm: 5.0,
learning-rate: 0.03, batch-size: 32

This experiment roughly simulates the disaster scenario, calculating the accuracy of the DPA-DP contribution compared to a disaster-unaware DP-SGD privacy setup. Using the AIDER dataset, we simulate 45 days (rounds) of normal operation with 100% normal images, followed by 5 days of disaster with 53% normal and 47% disaster images. The accuracy was calculated for the total set of labels, and separately for disaster labels.

The privacy budgeted consumption for the fixed DP-SGD and variable DPA-DP is shown below.

```
In [7]: epsilon_data = {}  
rounds=50  
noise_multiplier_switch = 45  
n_samples = 487  
noise_multiplier_1 = 0.4  
noise_multiplier_2 = 0.4  
epsilon_data["default"] = epsilon_curve(opacus.accountants.PRVAccountant(  
noise_multiplier_1 = 0.51  
noise_multiplier_2 = 0.26  
epsilon_data["dpa-dp"] = epsilon_curve(opacus.accountants.PRVAccountant()  
print(str((epsilon_data["default"][-1], epsilon_data["dpa-dp"][-1])))  
plot_epsilon(epsilon_data)
```

(122.20312621143057, 121.94853085314452)



```
In [8]: import pandas
dpadp_csv = pandas.read_csv("fixed_dpa-dp.csv")
last_round = dpadp_csv.loc[dpadp_csv.groupby("id")["round"].idxmax(), ["i
acc = dict(zip(last_round["id"], last_round["accuracy"]))
pandas.DataFrame([[acc["fixed"], acc["dpa-dp"]], [acc["fixed_sub"], acc["
columns=["fixed", "dpa-dp"], index=["total accuracy", "disaster accur
```

```
Out[8]:
```

	fixed	dpa-dp
total accuracy	0.681680	0.684780
disaster accuracy	0.004878	0.014634

DPA-DP early results and next steps

As it seems, for an equal privacy budget, the DPA-DP contribution shows better accuracy. The results are expected to improve. The epsilon will decrease to 1.0-5.0 after adding more images to the dataset. The DPA-DP accuracy will improve after extending the training to 90 days of normal operation followed by 10 days of disaster, multiplied by 24 (hourly evaluation), which will give 2,400 rounds and a better chance for accuracy to improve.

```
In [ ]:
```